



if / else

DECISION MAKING IN JAVA

Complete Study Guide with Examples & Flowcharts

if

if-else

Nested if

switch

B.Tech – Artificial Intelligence & Machine Learning

Karunya Institute of Technology and Sciences

Batch 2023 – 2027

TABLE OF CONTENTS

01	Introduction to Decision Making	3
02	if Statement	4
03	if-else Statement	7
04	Nested if Statement	10
05	switch Statement	13
06	Complete Program Examples	17
07	Quick Reference Summary	19

01

Introduction to Decision Making

How Java programs choose between different paths of execution



What is Decision Making?

Decision-making statements let a program choose different actions based on conditions. Java provides four main tools for this: if, if-else, nested if, and switch. Each evaluates a condition and decides which block of code should run, making programs dynamic instead of always executing the same fixed sequence.

The Four Decision-Making Tools

Statement	Use Case
if	Execute code only when ONE condition is true
if-else	Choose between TWO alternative paths
Nested if	Check a condition INSIDE another condition
switch	Choose ONE path from MANY fixed options

Real-Life Analogy: The Traffic Signal



Traffic Signal Decision Logic

if = checks if the light is green and allows movement. if-else = decides whether to move or stop based on the signal. Nested if = first checks if the light is green, then checks if the road is clear. switch = selects an action based on the signal color (Red, Yellow, or Green) directly.

02

if Statement

Execute a block of code only when a condition is true



What is the if Statement?

The if statement is the simplest decision-making tool in Java. It executes a block of code only when its condition evaluates to true. If the condition is false, the block is simply skipped and the program moves to the next statement.

Syntax

if – Syntax

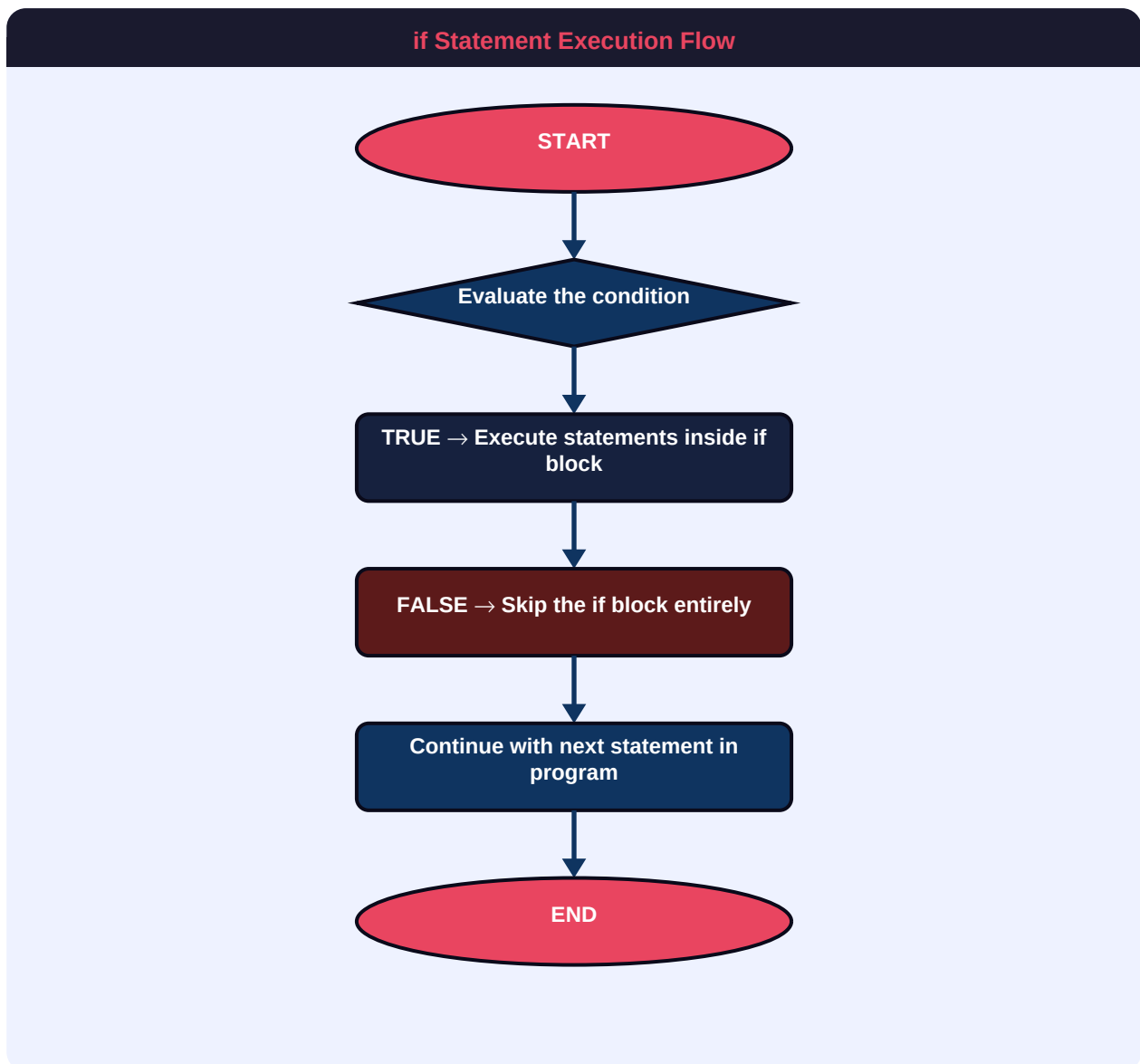
```
if (condition) {  
    // statements to execute  
}
```

Example – Voting Eligibility

VotingCheck.java

```
int age = 20;  
  
if (age >= 18) {  
    System.out.println("Eligible to vote");  
}  
  
// Since age (20) >= 18 is TRUE, this prints:  
// Eligible to vote
```

if Statement – Flowchart



Flow of Execution

Step	Action
1	The condition inside the if statement is evaluated
2	If the condition is true, the statements inside the block execute
3	If the condition is false, the block is skipped
4	Program continues with the next statement either way



Real-Life Analogy: Automatic Door

An automatic door at a shopping mall opens only if a person stands in front of the sensor. If no one is detected, the door remains closed. Similarly, an if statement executes its block only when the specified condition is true.

03

if-else Statement

Choose between two alternative paths of execution



What is the if-else Statement?

The if-else statement provides two possible paths. If the condition is true, the if block executes; otherwise, the else block executes. Exactly ONE of the two blocks always runs — never both, never neither.

Syntax

if-else – Syntax

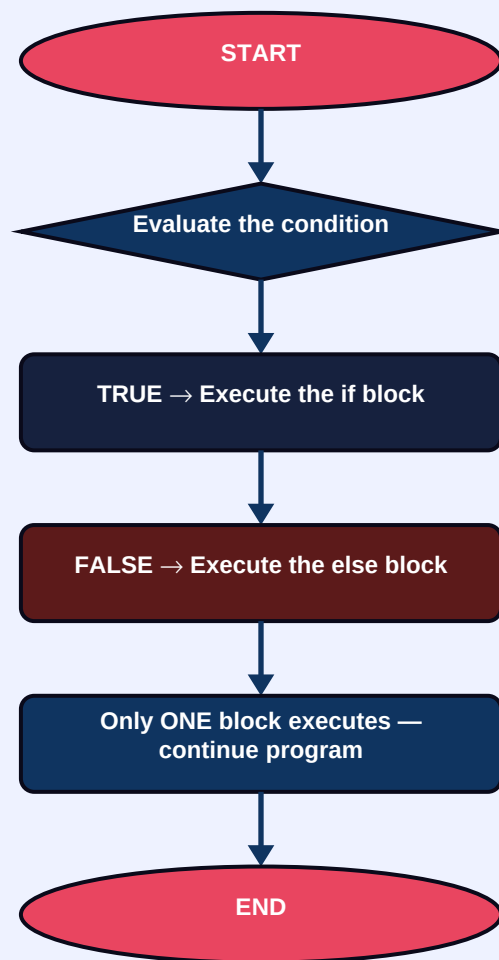
```
if (condition) {  
    // statements if condition is true  
} else {  
    // statements if condition is false  
}
```

Example – Pass or Fail

PassFailCheck.java

```
int marks = 45;  
  
if (marks >= 50) {  
    System.out.println("Pass");  
} else {  
    System.out.println("Fail");  
}  
  
// Since marks (45) >= 50 is FALSE, this prints:  
// Fail
```

if-else Statement – Flowchart

if-else Execution Flow (Two Paths)**Example – Voting Eligibility (with else)****VotingCheck2.java**

```
int age = 16;

if (age >= 18) {
    System.out.println("Eligible to vote");
} else {
    System.out.println("Not eligible to vote");
}

// Since age (16) is less than 18, this prints:
// Not eligible to vote
```



Real-Life Analogy: Movie Theater Entry

A movie theater has an age restriction. If your age is 18 or above, you are allowed to enter; otherwise, you are denied entry. Similarly, the if-else statement chooses one of two actions based on whether the condition is true or false.

04

Nested if Statement

An if statement placed inside another if statement



What is a Nested if?

A nested if is an if statement placed inside another if statement. It is used when multiple conditions need to be checked in sequence — the inner condition is only evaluated if the outer condition is true. Useful when one decision depends on the result of another decision.

Syntax

Nested if – Syntax

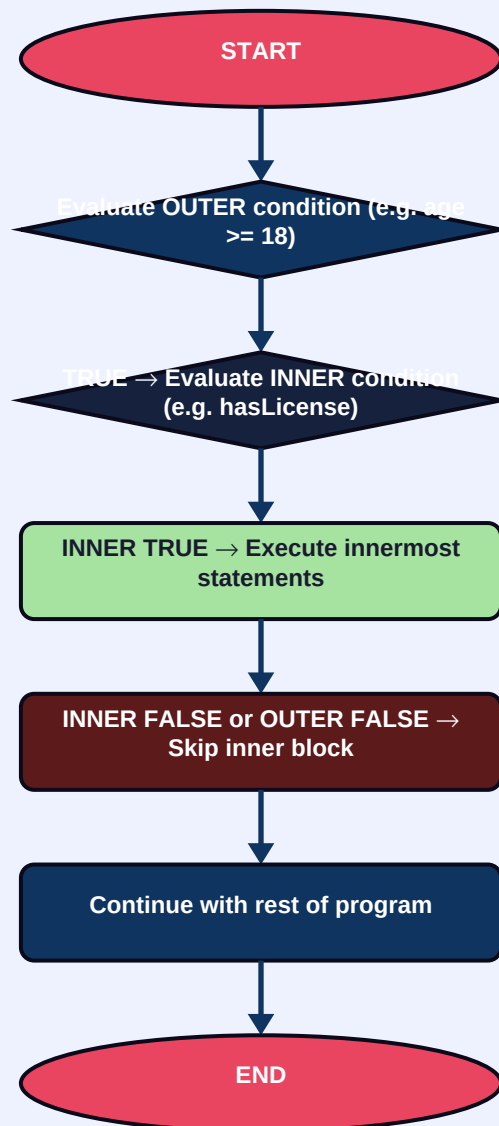
```
if (condition1) {  
    if (condition2) {  
        // statements - runs only if BOTH conditions are true  
    }  
}
```

Example – Driving Eligibility

DriveCheck.java

```
int age = 22;  
boolean hasLicense = true;  
  
if (age >= 18) {  
    if (hasLicense) {  
        System.out.println("Can drive");  
    }  
}  
  
// Outer condition true (age=22) AND inner condition true (hasLicense)  
// Output: Can drive
```

Nested if – Flowchart

Nested if Execution Flow (Two-Level Check)**Example – Scholarship Eligibility****ScholarshipCheck.java**

```
boolean passed = true;
int attendance = 95;

if (passed) {
    if (attendance > 90) {
        System.out.println("Eligible for Scholarship");
    }
}

// Both conditions true → Eligible for Scholarship
```

Complete Nested if Example (with else branches)

NestedIfDemo.java

```
public class NestedIfDemo {  
    public static void main(String[] args) {  
        int age = 20;  
        boolean hasLicense = true;  
  
        if (age >= 18) {  
            if (hasLicense) {  
                System.out.println("Eligible to drive.");  
            } else {  
                System.out.println("Driving license is required.");  
            }  
        } else {  
            System.out.println("Not eligible to drive.");  
        }  
    }  
}
```

**Real-Life Analogy: Secure Office Building**

Entering a secure office requires two checks. First, the security guard checks whether you have an ID card. If you have one, a second check verifies whether you have access permission to a specific room. Only when BOTH conditions are met are you allowed to enter — just like a nested if checking one condition inside another.

05

switch Statement

Select one block of code from multiple options



What is the switch Statement?

The switch statement selects one block of code from multiple alternatives based on the value of an expression. It is a cleaner alternative to multiple if-else statements when checking a single variable against many fixed values.

Syntax

switch – Syntax

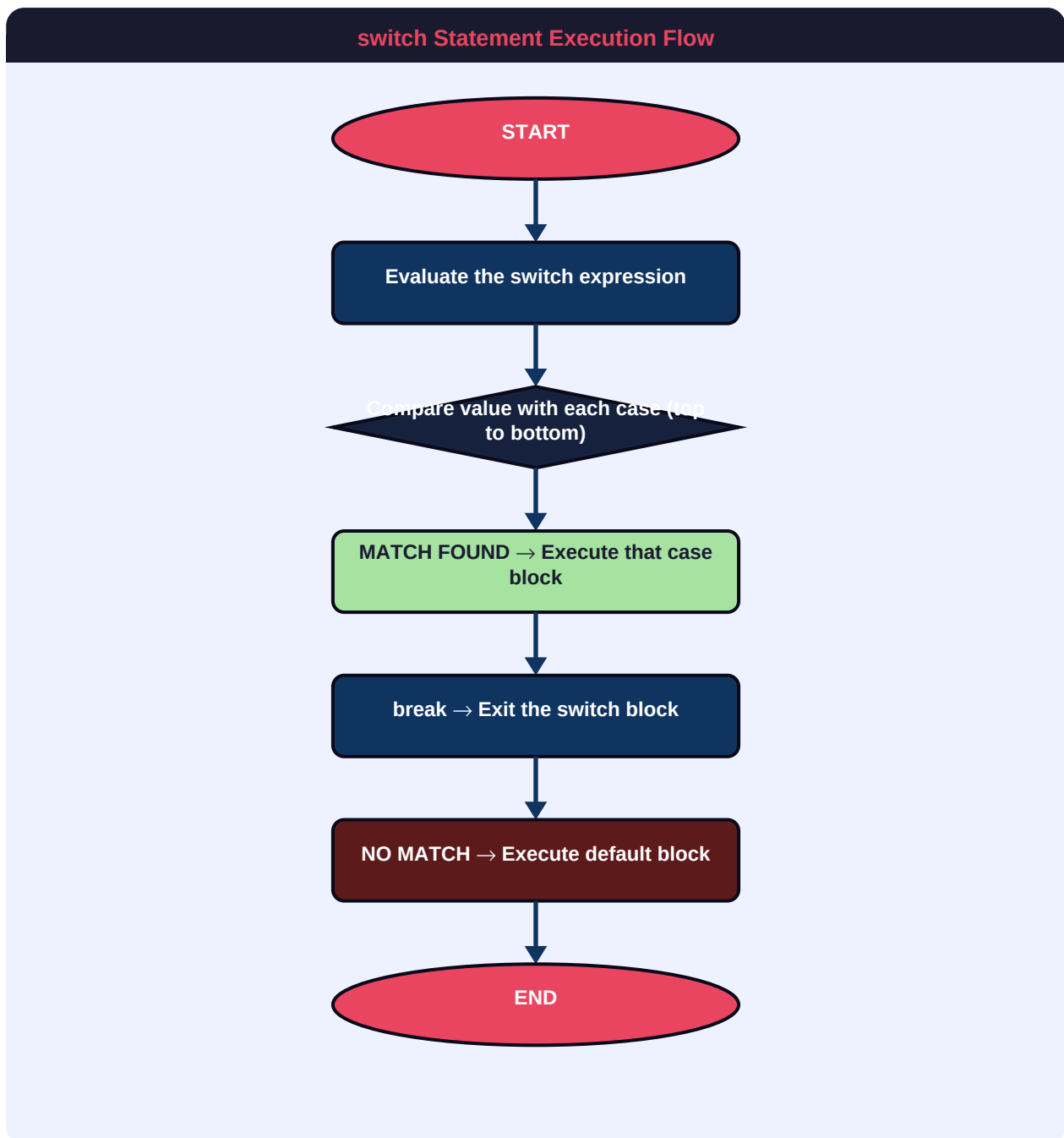
```
switch (expression) {  
    case value1:  
        // statements  
        break;  
    case value2:  
        // statements  
        break;  
    default:  
        // statements  
}
```

Example – Day of the Week

DayChecker.java

```
int day = 3;  
  
switch (day) {  
    case 1:  
        System.out.println("Monday");  
        break;  
    case 2:  
        System.out.println("Tuesday");  
        break;  
    case 3:  
        System.out.println("Wednesday");  
        break;  
    default:  
        System.out.println("Invalid Day");  
}  
  
// Since day = 3 matches case 3, output: Wednesday
```

switch Statement – Flowchart



Flow of Execution

Step	Action
1	The expression inside switch is evaluated
2	Its value is compared with each case from top to bottom
3	When a matching case is found, that block executes
4	break exits the switch block after execution
5	If no case matches, the default block executes



Real-Life Analogy: Vending Machine

A vending machine displays a drink based on the user's choice: press 1 for Coffee, 2 for Tea, 3 for Juice, any other number shows an error. Similarly, switch chooses one action from multiple predefined options based on a given value.

Important Note: Why break Matters

If **break** is omitted, execution 'falls through' into the next case regardless of whether it matches — this is called **fall-through behaviour**. Always include **break** unless fall-through is intentionally desired.

Complete switch Example – All Days of the Week

SwitchDemo.java

```
public class SwitchDemo {  
    public static void main(String[] args) {  
        int day = 3;  
        switch (day) {  
            case 1: System.out.println("Monday");    break;  
            case 2: System.out.println("Tuesday");   break;  
            case 3: System.out.println("Wednesday"); break;  
            case 4: System.out.println("Thursday");  break;  
            case 5: System.out.println("Friday");     break;  
            case 6: System.out.println("Saturday");  break;  
            case 7: System.out.println("Sunday");     break;  
            default: System.out.println("Invalid day");  
        }  
    }  
}
```

06

Complete Program Examples

Combining if, if-else, nested if, and switch together

Program 1: ATM Language Selector (switch)

ATMLanguage.java

```
import java.util.Scanner;

public class ATMLanguage {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Select language (1-English 2-Hindi 3-Tamil): ");
        int choice = sc.nextInt();

        switch (choice) {
            case 1:
                System.out.println("Language set to English");
                break;
            case 2:
                System.out.println("Language set to Hindi");
                break;
            case 3:
                System.out.println("Language set to Tamil");
                break;
            default:
                System.out.println("Invalid selection");
        }
        sc.close();
    }
}
```

Program 2: Traffic Signal Decision (Nested if + switch)

TrafficSignal.java

```
public class TrafficSignal {
    public static void main(String[] args) {
        String signal = "Green";
        boolean roadClear = true;

        switch (signal) {
            case "Red":
                System.out.println("Stop");
                break;
            case "Yellow":
                System.out.println("Slow down");
                break;
            case "Green":
                if (roadClear) {
                    System.out.println("Go - road is clear");
                } else {
                    System.out.println("Wait - road is busy");
                }
                break;
            default:
                System.out.println("Invalid signal");
        }
    }
}
```

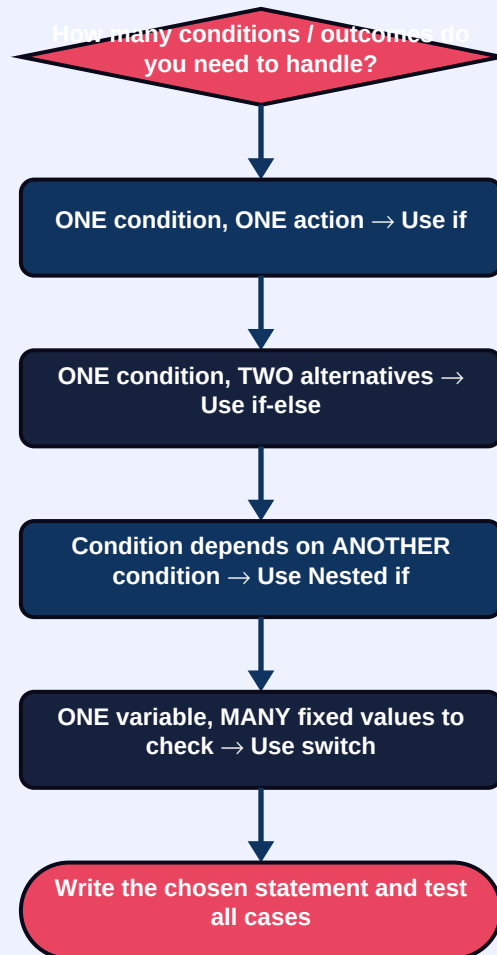

07

Quick Reference Summary

All decision-making concepts at a glance

Choosing the Right Decision Statement

Which Decision Statement Should I Use?



Key Takeaways

Statement	Core Idea	Key Syntax
if	Run code only if condition is true	<code>if (condition) { ... }</code>
if-else	Choose between two paths	<code>if (...) {...} else {...}</code>
Nested if	Check condition inside another condition	<code>if (...) { if (...) {...} }</code>
switch	Pick one path from many fixed values	<code>switch (x) { case v: ... break; }</code>

Real-Life Analogy Recap

Statement	Real-Life Analogy
if	Automatic door – opens only if a person is detected
if-else	Movie theater entry – allowed or denied based on age
Nested if	Secure office building – ID check, then room access check
switch	Vending machine / ATM language selector – pick from fixed options

■ **Pro Tip:** Use **if** for a single check, **if-else** for two outcomes, **nested if** when one decision depends on another, and **switch** when comparing one variable against many fixed values. Never forget **break** in a switch case unless fall-through is intentional!